

Agentic-Financial-Operations-Assistant

1. Executive Summary & Problem Statement

Financial operations (FinOps) teams handle critical, high-stakes workflows every day—including customer support queries, payment processing, refund claims, transaction disputes, fraud investigations, and core account operations.

Traditionally, these processes are heavily manual, fragmented, and prone to human error. Human operators are forced to context-switch across multiple legacy systems, manually cross-referencing customer profiles, transaction logs, risk databases, and regulatory compliance rules. This creates significant operational bottlenecks, delays resolution times, and introduces compliance risks.

Operational Challenge Example:

"Customer C1001 was charged twice ₹7,500. Please process the refund."

Required Execution Sequence:

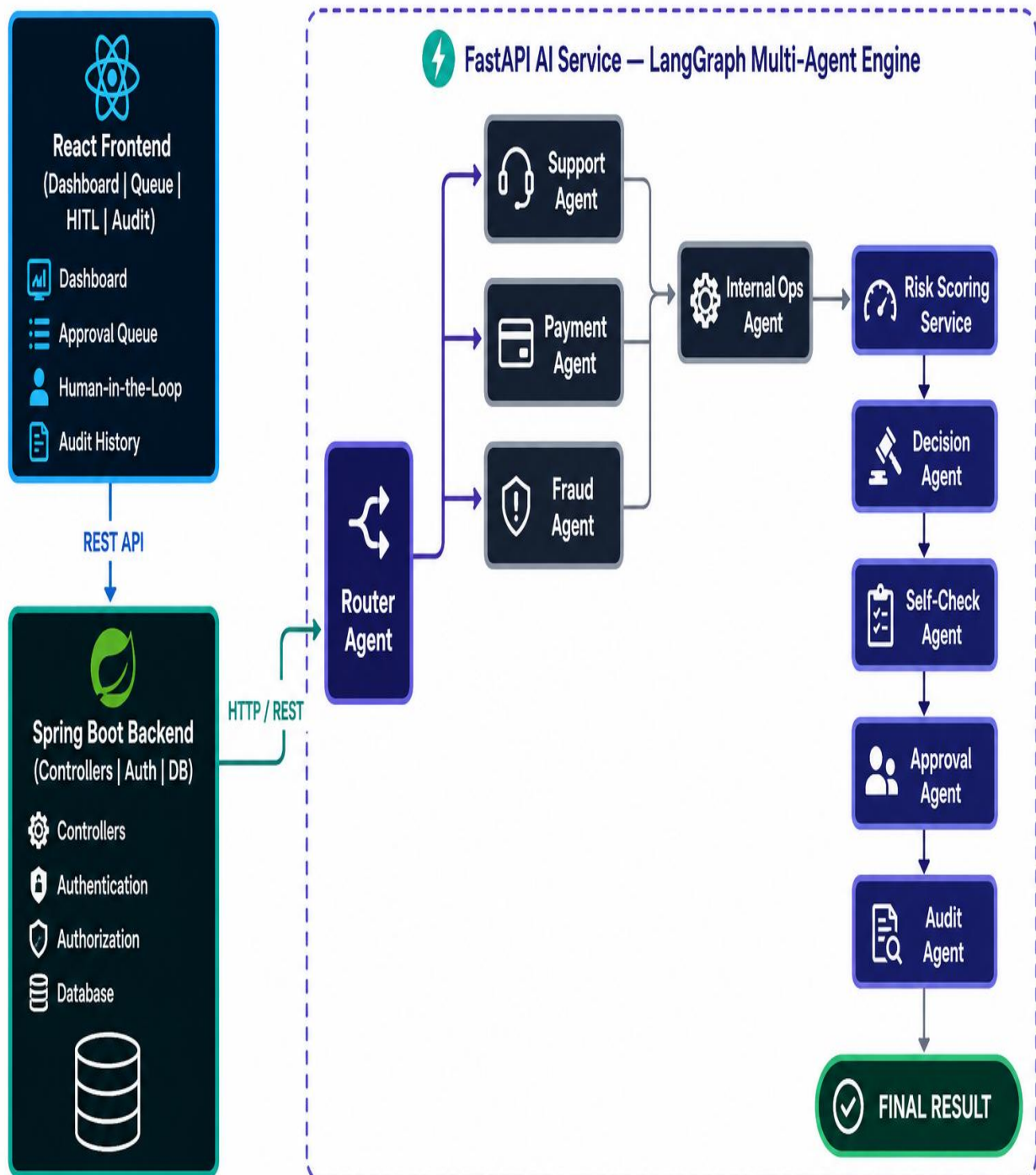
1. Customer Identification → Verify identity & loyalty tier
2. Support History Check → Review recent tickets & channel activity
3. Dispute History Analysis → Inspect chargebacks & ledger anomalies
4. Fraud Indicator Check → Screen device ID, location, & velocity
5. Risk Scoring → Calculate deterministic risk score (0-100)
6. Decision Logic → Synthesize agent reports into a decision
7. Human Approval Routing → Route to manager queue if threshold is met
8. Explainability Verification → Perform self-check against safety policies
9. Immutable Audit Logging → Record step-by-step reasoning for compliance

The Solution: Agentic Automation

The **Agentic Financial Operations Assistant** automates this end-to-end workflow by orchestrating autonomous, specialized AI agents using LangGraph. Operating under strict deterministic business logic, cost-optimization routing, and mandatory Human-in-the-Loop(HITL) safeguards for high-risk actions, the platform delivers speed without compromising financial control or security.

2. Enterprise System Architecture

The platform uses a 3-tier enterprise architecture separating the user interaction layer, business orchestration backend, and autonomous AI reasoning services.



3. Specialized Multi-Agent Architecture

Instead of relying on a single monolithic Large Language Model (LLM), the platform utilizes specialized, stateful AI agents managed via **LangGraph**.

Agent Name	Core Responsibility & Operations
1. Router Agent	Evaluates incoming prompts and dynamically routes execution only to required agents, reducing latency and LLM invocation costs.
2. Support Agent	Retrieves customer profiles, loyalty tiers, active complaints, support channels, and previous ticket history.
3. Payment Agent	Analyzes ledger history, failed transactions, duplicate billing indicators, chargeback records, and payment anomalies.
4. Fraud Agent	Evaluates velocity spikes, untrusted devices, geolocation anomalies, suspicious beneficiaries, and active fraud cases.
5. Internal Ops Agent	Manages core banking operational requests such as KYC updates, mobile/email updates, account freeze/unfreeze, and limits.
6. Decision Agent	Synthesizes reports from all prior agents to generate formal business decisions: APPROVE, RESOLVE, ESCALATE, or REJECT.
7. Self-Check Agent	Acts as an automated safety layer. Validates the decision against strict business rules before execution to prevent safety policy breaches.
8. Approval Agent	Governs Human-in-the-Loop (HITL) workflows. Automatically approves low-risk tasks while queueing high-risk/high-value actions for manager review.
9. Audit Agent	Captures full agent execution traces, decisions, reasoning, and timestamps into an immutable audit stream for regulatory compliance.

4. Deterministic Risk Scoring Framework

The platform computes an aggregate Risk Score (**0 to 100**) prior to decision execution using weighted behavioral and transactional flags:

Risk Flag Weightings & Tiers

Category	Risk Indicator Flag	Score Impact	Risk Score Range	Risk Level	Operational Action & Handling Protocol
Transaction Anomalies	Velocity Spike Detected	+30	0 – 20	Low Risk	Auto-Approve: Executed instantly without manual intervention.
Beneficiary Details	Unusual / High Beneficiary	+25	21 – 40	Moderate Risk	Standard Review: Passed through standard automated checks.
Device Security	New / Untrusted Device	+20	41 – 60	Medium Risk	Manager Review: Held in queue for low-priority manager review.
Financial Thresholds	High Transaction Amount Threshold	+20	61 – 80	High Risk	Mandatory HITL: Processing paused until manager explicitly approves.
Account Access	Failed Login / Security Attempts	+15	81 – 100	Critical Risk	Account Freeze: Blocked immediately and sent to fraud investigators.
Account History	Multiple Existing Disputes	+10	—	—	Adjusts total score upwards.
Trust Signal	Verified / Trusted Device	-15	—	—	Reduces total score.

5. Shared Agent State & Schema

LangGraph agents communicate across the workflow pipeline by updating a single shared state schema:

JSON {

```
"case_id": "729F9701",
"customer_id": "C1001",
"account_id": "A2001",
"request_type": "refund",
"amount": 7500,
"description": "Duplicate charge",
"support_summary": "Gold tier customer with 2 recent inquiries.",
"payment_summary": "Duplicate transaction detected; 1 prior dispute.",
"fraud_summary": "Risk score: 20/100. No velocity spikes or untrusted devices.",
"risk_score": 20,
"risk_flags": [],
"decision": "APPROVE",
"action": "issue refund pending manager approval",
"approval_required": true,
"approval_status": "PENDING",
"self_check_passed": true,
"reason": "Refund exceeds automatic threshold (₹1,000) but satisfies low fraud risk constraints.",
"agent_trace": [
  "router_agent",
  "support_agent",
  "payment_agent",
  "decision_agent",
  "self_check_agent",
  "approval_agent",
  "audit_agent"
],
"audit_log": [] }
```

6. Project Repository Directory Structure

Here is the repository structure organizing the multi-service codebase across FastAPI, Spring Boot, and React components:

```
AI-BANKING-AGENT/
├── .github/                # CI/CD Workflows & GitHub Actions
├── .pytest_cache/         # Python Test Cache
├── agents/                # LangGraph Agent Implementations
│   ├── __init__.py
│   ├── router_agent.py    # Intelligent Request Router
│   ├── support_agent.py   # Customer Profile & Ticket Lookup
│   ├── payment_agent.py   # Transaction Ledger & Dispute Analysis
│   ├── fraud_agent.py     # Security & Behavioral Risk Agent
│   ├── internal_ops_agent.py # Banking Workflow & KYC Operations
│   ├── decision_agent.py  # Primary Business Recommendation Engine
│   ├── self_check_agent.py # Deterministic Safety Verification
│   ├── approval_agent.py  # Human-In-The-Loop Approval Manager
│   └── audit_agent.py     # Compliance & Traceability Logger
├── api/                   # FastAPI REST Endpoints
│   ├── __init__.py
│   └── routes.py          # Operation Ingestion & Workflow Triggers
├── Backend/               # Enterprise Spring Boot Application
│   ├── src/main/java/com/banking/
│   │   ├── controllers/   # Banking & Approval REST Controllers
│   │   ├── services/      # Orchestration & FastAPI Client
│   │   ├── models/        # Domain Entities & DTOs
│   │   └── repository/    # PostgreSQL Database Access Layer
│   └── pom.xml            # Maven Build File
├── data/                  # MVP Datasets & Mock Registries
│   ├── customers.json
│   ├── transactions.json
│   ├── tickets.json
│   ├── fraud_cases.json
│   └── internal_workflows.json
├── docs/                  # Technical Specs, Specs & Design Diagrams
├── frontend/              # React + Vite Enterprise Banking Dashboard
│   ├── src/
│   │   ├── components/    # Agent Timeline, Risk Badges, Forms
│   │   ├── pages/         # Operations, Approvals, Audit Dashboard
│   │   └── services/      # Axios API Client
│   ├── package.json
│   └── vite.config.js
├── graph/                 # LangGraph Workflow Orchestration
│   ├── __init__.py
│   └── workflow.py        # Graph Compilation & State Transitions
├── logs/                  # File Appender Audit Trail ('audit.log')
├── models/                # Pydantic Schemas & Shared State Definitions
│   └── state.py
├── prompts/               # System & Few-Shot Prompts
│   ├── support_prompt.txt
│   ├── payment_prompt.txt
│   ├── fraud_prompt.txt
│   ├── decision_prompt.txt
│   └── self_check_prompt.txt
├── services/              # Domain Integration Services
│   ├── customer_service.py
│   ├── payment_service.py
│   ├── fraud_service.py
│   ├── audit_service.py
│   ├── prompt_service.py
│   └── risk_score_service.py
├── static/                # Static Assets & Generated Artifacts
└── tests/                 # Comprehensive Unit & Scenario Test Suite
```

```
├── utils/                 # Model Switching & Utility Handlers
│   └── llm.py             # Dual Fast/Reasoning LLM Routing Architecture
├── venv/                  # Python Virtual Environment
│   ├── .env               # Local Environment Configuration
│   ├── .env.example       # Template Configuration File
│   └── docker-compose.yml  # Container Deployment Config
├── main.py                # FastAPI Application Entrypoint
├── README.md              # GitHub Repository Documentation
└── requirements.txt        # Python Dependencies List
```

7. End-to-End Operational Scenarios

Scenario	Input Request	Agent Execution Trail	Decision & HITL Outcome
1. Low-Risk Refund	<i>"Refund ₹500 to C1004 for a failed UPI transaction."</i>	Router → Payment → Decision → Self-Check → Approval	APPROVE / AUTO_APPROVED Refund executed immediately without human intervention.
2. High-Value Refund	<i>"Customer C1001 was charged twice ₹7,500. Process refund."</i>	Router → Support → Payment → Decision → Self-Check → Approval	APPROVE / PENDING Queued for Manager Review due to amount exceeding threshold.
3. Fraud Investigation	<i>"Investigate suspicious IMPS transfer of ₹125,000 on account A4001."</i>	Router → Payment → Fraud → Decision → Self-Check → Approval	ESCALATE / PENDING Risk Score 85/100. Account hold placed pending Fraud Review.

8. Setup & Installation Guide

1. Run AI FastAPI Service

Bash

```
python -m venv venv
```

```
venv\Scripts\activate    # On Windows
```

```
pip install -r requirements.txt
```

```
uvicorn main:app --reload --port 8000
```

2. Run Spring Boot Backend

Bash

```
cd Backend
```

```
mvn clean install
```

```
mvn spring-boot:run    # Running on port 8080
```

3. Run React Frontend

Bash

```
cd frontend
```

```
npm install
npm run dev
```

9. Conclusion

The **Agentic Financial Operations Assistant** transforms complex financial operations into an automated, transparent, and secure workflow. By combining specialized Multi-Agent orchestration via LangGraph with a deterministic Risk Scoring Framework, the system effectively automates routine, low-risk requests while ensuring strict regulatory safety through Human-in-the-Loop (HITL) approvals for high-risk actions.

Key Takeaways

- **Operational Efficiency:** Dramatically cuts manual verification time for support, payments, and internal ops workflows.
- **Enterprise Safety:** Combines rule-based risk evaluation, self-check verification, and mandatory manager sign-offs to eliminate unauthorized executions.
- **Full Auditability:** Maintains an immutable trace of every agent reasoning step, risk score, and decision for complete compliance readiness.

